

Q.1) Read the current directory and display the name of the files, no of files in current directory

[10 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <dirent.h>

int main() {

    DIR *dir;

    struct dirent *entry;

    int file_count = 0;


    dir = opendir(".");

    if (!dir) {

        perror("opendir");

        return 1;

    }


    printf("Files in current directory:\n");

    while ((entry = readdir(dir)) != NULL) {

        if (entry->d_type == DT_REG) { // Only regular files

            printf("%s\n", entry->d_name);

            file_count++;

        }

    }


    printf("\nTotal number of files: %d\n", file_count);


    closedir(dir);

    return 0;

}
```

Q.2) Write a C program to implement the following unix/linux command (use fork, pipe and exec system call). Your program should block the signal Ctrl-C and Ctrl-\ signal during the execution.

i. `ls -l | wc -l` [20 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <signal.h>

#include <sys/wait.h>


int main() {

    int fd[2];

    pid_t pid1, pid2;


    // Block Ctrl-C and Ctrl-\
    signal(SIGINT, SIG_IGN);
    signal(SIGQUIT, SIG_IGN);


    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }


    pid1 = fork();
    if (pid1 < 0) {
        perror("fork");
        exit(1);
    }


    if (pid1 == 0) {
        // First child: ls -l
        close(fd[0]); // Close read end
        dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe
        close(fd[1]);
```

```
    execlp("ls", "ls", "-l", NULL);  
    perror("execlp ls");  
    exit(1);  
}
```

```
pid2 = fork();  
if (pid2 < 0) {  
    perror("fork");  
    exit(1);  
}
```

```
if (pid2 == 0) {  
    // Second child: wc -l  
    close(fd[1]);      // Close write end  
    dup2(fd[0], STDIN_FILENO); // Redirect stdin from pipe  
    close(fd[0]);
```

```
  
    execlp("wc", "wc", "-l", NULL);  
    perror("execlp wc");  
    exit(1);  
}
```

```
// Parent closes both ends  
close(f
```